Top-Down Analyse für LL(1)-Grammatiken

```
push(S)              /* Startsymbol S auf Stack              */
a:=nexttoken()       /* Eingabesymbol := erstes Symbol */
repeat
        if Stack leer and a=EOF
        then accept                    /* erfolgreich beendet*/
        else X:= oberstes Stackelement
            if X = a then              /* ”Match” eines Terminalsymbols */
                pop()
                a:=nexttoken()
            elseif ( X ∈ N and TAB[X, a] = (X → Y_1 ... Y_k) ) then
                print(X → Y_1 ... Y_k)   /* Ausgabe des nächsten Ableitungsschritts*/
                pop()
                push(Y_k) , . . . , push(Y_1)
                else
                        Fehlerbehandlung
        end if end if
end repeat
```

Beispiel

Eingabe: accbbdd

$S \rightarrow aA$
$A \rightarrow B \mid cAd \mid \varepsilon$
$B \rightarrow bb$

	a	b	c	d	$
S	S→aA				
A		A→B	A→cAd	A→ε	A→ε
B		B→bb			

Stack	restliche Eingabe	Aktion
S	accbbdd	S → aA
aA	accbbdd	pop
A	ccbbdd	A→cAd
cAd	ccbbdd	pop
Ad	cbbdd	A→cAd
cAdd	cbbdd	pop
Add	bbdd	A→B
Bdd	bbdd	B→bb
bbdd	bbdd	pop
bdd	bdd	pop
dd	dd	pop
d	d	pop
	$	accepted

Schlussfolgerungen über den Top-Down Parser

- Wenn G eine LL(1)-Grammatik ist und $w \in \mathrm{T}^*$, dann gilt:
  - (i) Der Algorithmus (siehe Vorseite) terminiert
  - (ii) Wenn der Algorithmus angewandt auf w mit leerem Keller und vollständig gelesenem Wort w terminiert, dann ist $w \in L(G)$
  - (iii) Wenn der Algorithmus w nicht akzeptiert, dann ist $w \notin L(G)$

– Der Algorithmus benötigt $\mathcal{O}(|w|)$ Schritte, um das Wort w zu analysieren.

Umformung von EBNF in eine kontextfreie Grammatik

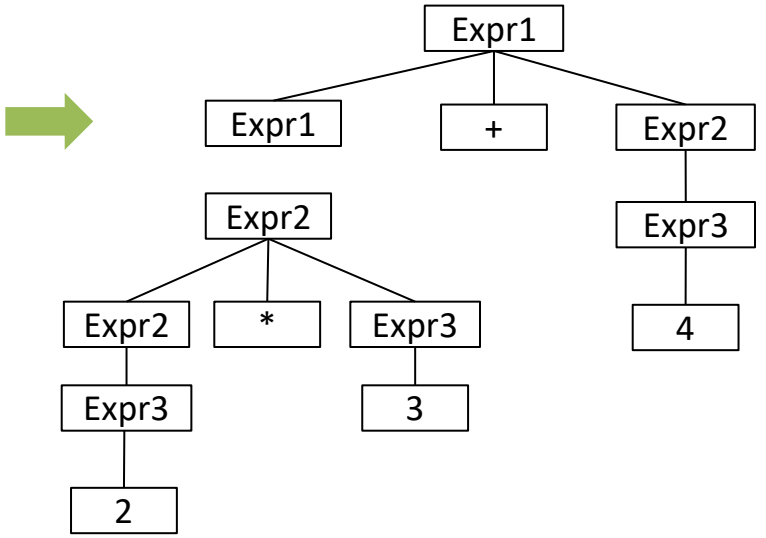
	EBNF	KFG
Alternative	$A ::= B \mid C \mid \ldots$	$A \rightarrow B$ $A \rightarrow \mathrm{C}$ $\ldots$
Optional	$A ::= B\ [C]$	$A \rightarrow B$ $A \rightarrow \mathrm{B\ C}$
Wiederholung	$A ::= B\ \{\ C\ \}$	$A ::= B\ CList$ $CList \rightarrow C\ CList \mid \epsilon$

Semantische Aktionen

Beispiel:

$$Expr_1 \rightarrow Expr_1 + Expr_2 \mid Expr_2$$
$$Expr_2 \rightarrow Expr_2 * Expr_3 \mid Expr_3$$
$$Expr_3 \rightarrow true \mid false \mid Number$$

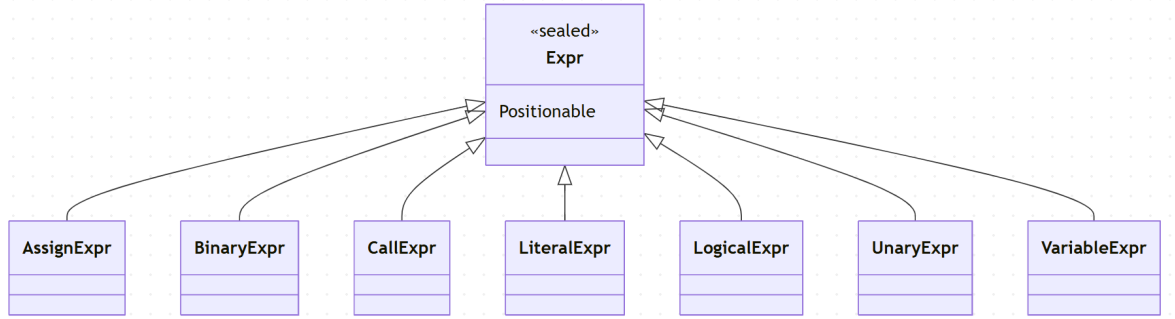
Eingabetext: 2*3+4

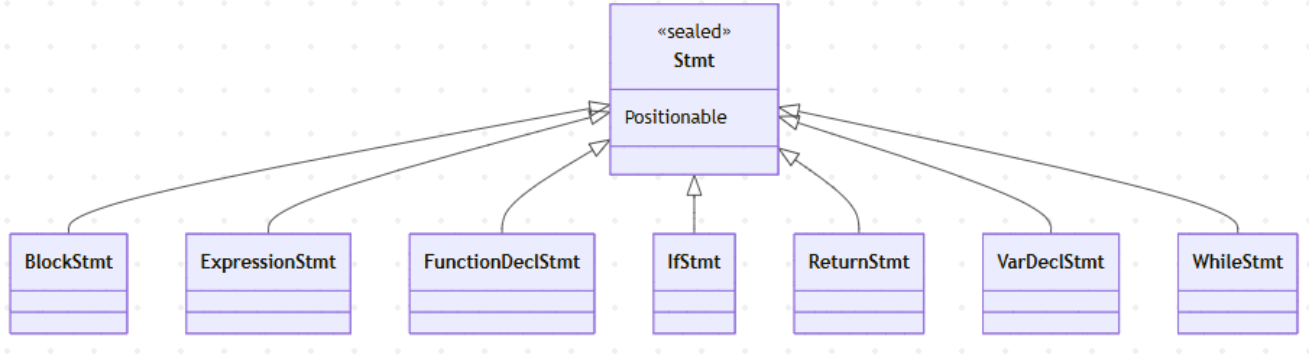


Eine Datenstruktur (Baum), die das erkannte TiEL-Programm abbildet.
Beobachtung: Die Hierarchie Expr1/Expr2/Expr3 im Baum ist redundant!

Abstrakter Syntaxbaum (AST)

- Schlüsselwörter, Klammern, etc. dienen nur der Erkennung der Syntax und werden danach nicht mehr gebraucht.
- Daher sprechen wir von „abstraktem“ Syntaxbaum (AST).
- Im TiEL-Interpreter besteht der AST aus 2 Arten Knoten:
  - Expr
  - Stmt



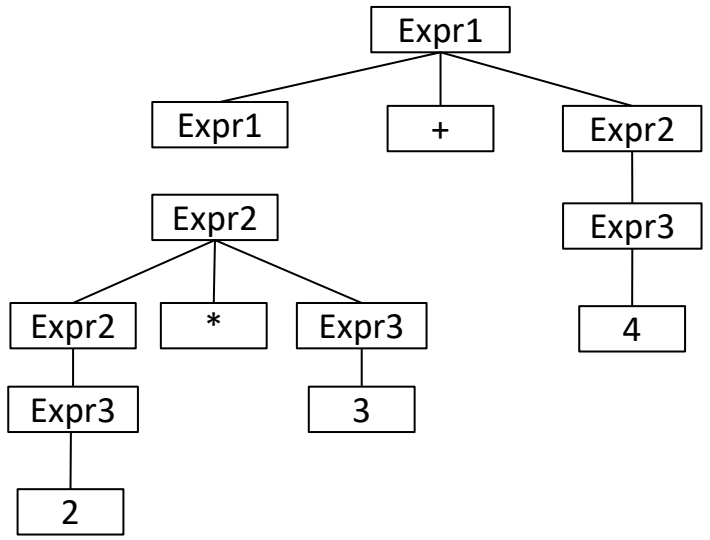


$$Expr1 \rightarrow Expr1 + Expr2 \mid Expr2$$
$$Expr2 \rightarrow Expr2 * Expr3 \mid Expr3$$
$$Expr3 \rightarrow true \mid false \mid Number$$

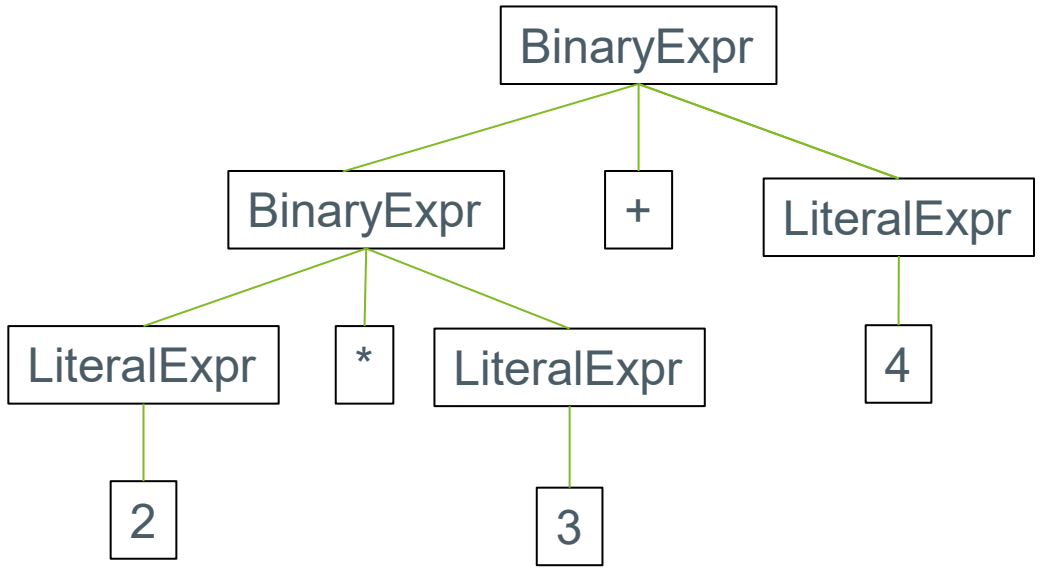
Beispiel 2*3+4

Damit ergibt sich für den Beispielausdruck der folgende Ableitungsbaum und AST:

Ableitungsbaum



AST



Der AST wird vom Compiler weiterbenutzt.

Linksassoziative Operatoren benötigen aber Linksrekursion (VL 6, Folie 19)

- Regeln wie $Expr1 \rightarrow Expr1 + Expr2 \mid Expr2$ stellen sicher, dass linksassoziative Operatoren (hier $+$) richtig behandelt werden (siehe Beispiel VL6).
- LL(1)-Grammatiken dürfen aber nicht linksrekursiv sein!
- Wiederholung:

Elimination von Linksrekursion
$X \rightarrow X\,\alpha \mid \beta$
kann man umschreiben in:
$X \rightarrow \beta\, X'$
$X' \rightarrow \alpha\, X' \mid \varepsilon$

Beispiel:

$$\underbrace{Expr1}_{X} \rightarrow \underbrace{Expr1}_{X} \underbrace{+ Expr2}_{\alpha} \mid \underbrace{Expr2}_{\beta}$$



$$\underbrace{Expr1}_{X} \rightarrow \underbrace{Expr2}_{\beta}\ \underbrace{Expr1Tail}_{X'}$$

$$\underbrace{Expr1Tail}_{X'} \rightarrow \underbrace{+Expr2}_{\beta}\ \underbrace{Expr1Tail}_{X'} \mid \epsilon$$

Praktikumsaufgabe 2: Umsetzen der Grammatik in Code

Beispiel: Unterste Ebene von Expressions E9:

$$E9 \rightarrow true|false|nil|Number|String|Identifier| \ldots$$

Wird im Code wie folgt definiert:

```
var e9 = parser.nonTerminal("e9");    // e9 ist ein Nichtterminal
…
e9.produces(TRUE)                     // E9 → true
  .withSemantic($ -> new LiteralExpr(TiELValue.TRUE));  // erzeuge eine Semantische Aktion
```

Alle Terminalzeichen sind schon in `de.thm.asc.tiel.interpreter.scanning.TokenType` definiert.

withSemantic

`withSemantic` verbindet eine Produktionsregel mit der Funktion, die den AST-Knoten aus den geparsten Teilen erzeugt.

Beispiel:

```
ifStatement.produces(IF, expression, block, elseOpt)
           .withSemantic($ -> new IfStmt($.get(1), $.get(2), $.get(3)));
```

`$.get(1)` ist das zweite (!) Element auf der rechten Seite der Produktion, also der Knoten, der beim Parsen der Expression aufgebaut wird, usw.

Listen (z.B. StmtList)

$StmtList \rightarrow Stmt\ \ StmtList \mid \epsilon$

Im Code:

```
statementList.produces(statement, statementList)
            .withSemantic($ -> cons($.get(0), $.get(1)));

statementList.produces()
            .withSemantic($ -> new ArrayList<Stmt>()); // epsilon
```

Also: Leere Liste mit `new ArrayList<T>()` erzeugen; `cons` fügt Head und Tail einer Liste zusammen.

Beim Parsen wird von innen nach außen aufgebaut (rekursiv), die Liste entsteht beim Zurücklaufen.

Elimination von Linksrekursion

Nach der Ersetzung der Linksrekursion durch **Rechtsrekursion** bleibt das Problem, dass die Operatoren **linksassoziativ** sind. Der Baum würde also verkehrt herum aufgebaut.

$E_5 \ -> E_6 \ E_5 Tail$
$E_5 \text{Tail} \ \rightarrow \epsilon| + E_6 \ E_5 Tail$

Lösung: Die semantischen Aktionen für Expressions liefern **Funktionen** zurück, die später richtig herum verkettet werden.

```
interface UnfinishedExpression extends Function<Expr, Expr> {
    static UnfinishedExpression identity() {
        return e -> e;
    }

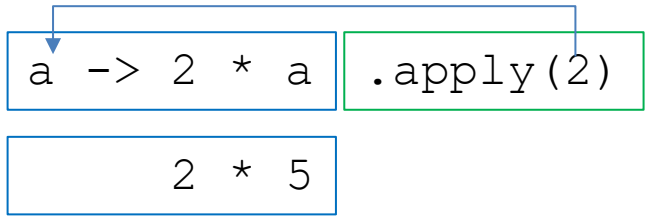
    static UnfinishedExpression of(Function<Expr, Expr> f) {
        return f::apply;
    }

    static UnfinishedExpression chain(Function<Expr, Expr> cons,
                                      Function<Expr, Expr> tail) {
        return UnfinishedExpression.of(l -> tail.apply(cons.apply(l)));
    }
}
```

Kurze Wiederholung Lambda-Funktionen

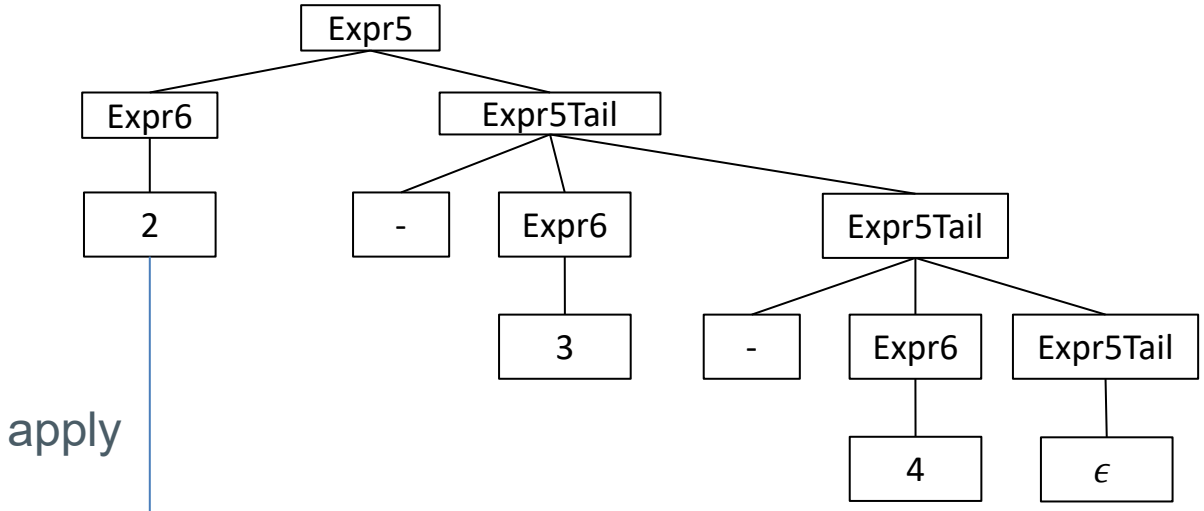
Seit Java 8 kann man in Java Lambda-Funktionen benutzen:

```
Function<Integer, Integer> theFunction = a -> 2*a;
System.out.printLn(theFunction.apply(5));  // Ergebnis: 10
```



`f.andThen(g)` entspricht g(f(x)) (erst f, dann g) – das entspricht im unserem Code `UnfinishedExpression.chain()`

Beispiel: 2 – 3 - 4



apply

$l \rightarrow new\ BinExp(l, -, 4)$

$l \rightarrow new\ BinExp(l, -, 3)$

chain

$l \rightarrow new\ BinExp(new\ BinExp(l, -, 3), -, 4)$

chain-Ergebnis

$new\ BinExp(new\ BinExp(2, -, 3), -, 4)$

Endergebnis

Beispiele für UnfinishedExpression

$E_5 \ -> E_6 \ E_5Tail$
$E_5\text{Tail} \ \rightarrow \epsilon| + E_6 \ E_5Tail$

```
e5.produces(e6, e5tail)    // E5 -> E6 E5Tail
    .withSemantic($ -> ((UnfinishedExpression) $.get(1)).apply($.get(0)));

e5tail.produces()
    .withSemantic($ -> UnfinishedExpression.identity());

e5tail.produces(PLUS, e6, e5tail)
    .withSemantic($ -> UnfinishedExpression.chain(
        l -> new BinaryExpr(l, BinaryExpr.Operator.ADD, $.get(1)),   // cons
        $.get(2)                                                     // tail
    ));
```

Wie benutzt man das? (1/4)

1. Ein Nichterminal A leitet direkt auf ein Nichtterminal B ab ($A \rightarrow B$).*
   Hier wird das Ergebnis von B einfach weitergeleitet an A: `a.produces(b);`
   Beispiel: `statement.produces(ifStatement);`

2. Zu dem Nichtterminal gibt es einen passenden Konstruktor:
   Verwende `.withSemantic(ClassName.class)`
   Beispiel:
   ```
   whileStatement.produces(WHILE, expression, block)
                 .withSemantic(WhileStmt.class);
   ```

   `withSemantic` erzeugt hier ein Objekt der Klasse und füllt den Konstruktor mit den Werten der rechten Seite der Produktion. Alternativ kann man schreiben:

   ```
   whileStatement.produces(WHILE, expression, block)
                 .withSemantic($ -> new WhileStmt($.get(1), $.get(2)));
   ```

Wie benutzt man das? (2/4)

3. Zugriff auf bestimmte Elemente
   Verwende `withSemantic($-> … $.get(i)…)`

```
Beispiel: e9.produces(LEFT_PAREN, expression, RIGHT_PAREN)
              .withSemantic($ -> $.get(1));
```

4. Transformation eines oder mehrerer Parameter

```
Beispiel: e7.produces(NOT, e8)
              .withSemantic($ -> new UnaryExpr(UnaryExpr.Operator.NOT, $.get(1)));
```

Wie benutzt man das? (3/4)

5. Rekursiv Listen aufbauen

   Beispiel: $statementList \rightarrow statement\ statementList \mid \epsilon$

```
statementList.produces(statement, statementList)
             .withSemantic($ -> cons($.get(0), $.get(1)));  // head tail

statementList.produces().withSemantic($ -> new ArrayList<Stmt>()); // epsilon
```

Wie benutzt man das? (4/4)

6. Token-Werte extrahieren
   Hier verwendet man `$.get(i)` - Tokens werden automatisch zu Werten konvertiert
   Beispiele:

```
e9.produces(IDENTIFIER).withSemantic($ -> new VariableExpr($.get(0)));
e9.produces(NUMBER)
  .withSemantic($ -> new LiteralExpr(new TiELValue.TNumber($.get(0))));
```

7. Token extrahieren
   Beispiel: $cmpOperator \rightarrow$ '<='

```
cmpOperator.produces(LESS_EQUAL).withSemantic($ -> BinaryExpr.Operator.LE);
```

8. Um linksassoziative Operatoren in rechtsrekursiven Grammatiken zu behandeln, nutze `UnfinishedExpression` – siehe Beispiel!